Administration Serveur Web

Modules disponibles :

- Pré-requis : Environnement de TP
- Gérer les services avec Systemd
- Planifier avec Crontab
- Serveur web NGINX
- PHP & PHP-FPM

Pré-requis

L'Environnement de TP

OBJECTIFS

- Comprendre pourquoi Docker a besoin d'une image spécifique
- Création de l'image (Dockerfile)
- TP : Initialisation du Lab

1. Introduction : Pourquoi un conteneur ?

Pour manipuler **systemd** et **systemctl**, nous avons besoin d'un OS Linux classique où systemd est le processus principal (PID 1).

Le problème avec Docker :

- Docker est conçu pour faire tourner un seul processus par conteneur (souvent l'application).
- Il n'utilise pas systemd par défaut pour gérer ses services internes.

La solution :

- Nous allons construire une image sur mesure basée sur Debian, où nous forcerons `systemd` à prendre la place du processus principal (`PID 1`).

2. Installation de l'Image Docker

Étape 1 : Créez un fichier `Dockerfile`

```
FROM debian:bookworm-slim
ENV DEBIAN_FRONTEND=noninteractive

# Installation des dépendances dont systemd
RUN apt-get update && apt-get install -y systemd systemd-sysv dbus && apt-get clean

# Nettoyage des cibles inutiles en conteneur
RUN rm -f /lib/systemd/system/multi-user.target.wants/* \
    /etc/systemd/system/*.wants/* /lib/systemd/system/local-fs.target.wants/* \
    /lib/systemd/system/sockets.target.wants/*udev* \
    /lib/systemd/system/sockets.target.wants/*initctl* \
    /lib/systemd/system/sysinit.target.wants/*

CMD ["/lib/systemd/systemd"]
```

Étape 2 : Compilez l'image

```
docker build -t debian-systemd .
```

3. Fonctionnement : Les Privilèges

Pour que `systemd` puisse gérer correctement les services, il a besoin d'interagir avec les composants bas niveau du noyau Linux (les cgroups).

C'est pourquoi nous devrons lancer notre conteneur avec des paramètres de sécurité assouplis :

1. `--privileged` : Donne au conteneur quasiment les mêmes droits que l'hôte.
2. `-v /sys/fs/cgroup:/sys/fs/cgroup:rw` : Monte le système de fichiers virtuel des cgroups en lecture/écriture, indispensable pour que systemd puisse suivre les processus.

4. Sécurité : Les Fondations

Avant même d'installer quoi que ce soit, un serveur Linux doit être sécurisé dès la première minute.

1. L'Accès SSH

- **Le Risque :** Des robots scannent internet 24/7 pour tester des mots de passe sur le port 22 (SSH).
- **La Solution :** Dans `/etc/ssh/sshd_config` :
  - Désactiver la connexion avec mot de passe : `PasswordAuthentication no`
  - Exiger une clé cryptographique (RSA/Ed25519).
  - Interdire formellement à `root` de se connecter directement : `PermitRootLogin no`.

2. Le Pare-Feu (Firewall)

- **Le Risque :** Laisser tous les ports d'un serveur ouverts sur internet est suicidaire. Un service mal configuré (comme une BDD interne) devient piratable de l'extérieur.
- **La Solution :** Toujours activer un pare-feu (ex: `ufw` sur Ubuntu/Debian).
  - `ufw default deny incoming` (On bloque tout par défaut).
  - `ufw allow ssh` (On autorise notre propre connexion !).
  - `ufw enable` (On active la forteresse).

5. TP Basique : Initialisation du Lab

Objectif : Démarrer le conteneur qui servira pour le reste du cours.

1. **Lancement du conteneur**
   - Lancez le conteneur en arrière-plan (`-d`) avec les droits requis.
   - Nommez le conteneur `lab-systemd` .
2. **Vérification**
   - Entrez dans le conteneur avec un shell interactif (`bash`).
   - Vérifiez que systemd tourne correctement avec la commande `systemctl status` .

Module Systemd

Maîtriser systemctl et les démons sous Linux

AU PROGRAMME

- Comprendre le rôle de systemd et systemctl
- Connaître les pièges liés à Docker
- Créer et gérer son propre service
- TP : Création d'un service basique robuste

1. Qu’est-ce que Systemd & Systemctl ?

Systemd est le système d’initialisation standard de la majorité des distributions Linux modernes (Debian, Ubuntu, CentOS…). Il est le premier processus démarré par le noyau (PID 1).

À quoi sert-il ?

- Démarrer les services (démons) au boot de la machine.
- Surveiller les processus et les redémarrer en cas de crash.
- Gérer les dépendances entre les services (ex: ne démarrer l’app que si la base de données est prête).
- Centraliser les logs via `journalctl`.

Comment interagir avec ? On utilise la commande `systemctl`.

```
systemctl status <service>  # Voir l'état
systemctl start <service>   # Démarrer
systemctl stop <service>    # Arrêter
systemctl restart <service> # Redémarrer
systemctl enable <service>  # Lancer au démarrage de la machine
```

2. Installation de Systemd

Contrairement à de nombreux services (comme Nginx ou MySQL), **systemd est pré-installé et actif par défaut** sur la quasi-totalité des distributions Linux modernes (Debian, Ubuntu, CentOS, RHEL, Fedora…).

Vous n'avez donc généralement rien à installer.

Comment vérifier que systemd est bien votre gestionnaire d'init ?

1. Vérifiez le processus numéro 1 :

```
ps -p 1 -o comm=
```

(Si cela retourne `systemd`, vous êtes au bon endroit !)

2. Vérifiez la version installée :

```
systemctl --version
```

2. Les pièges avec Docker

⚠️ **Systemd vs Docker : Une philosophie différente**

- **Le piège classique :** Vouloir exécuter `systemctl start mon-service` à l'intérieur d'un conteneur Docker. **Résultat :** `System has not been booted with systemd as init system (PID 1). Can't operate.`
- **Pourquoi ?** Docker est conçu pour exécuter **un seul processus par conteneur** au premier plan. Il n'a pas (par défaut) de PID 1 "Systemd" pour gérer des services en arrière-plan.
- **La solution :** Dans Docker, c'est le moteur Docker qui agit comme "gestionnaire de service". L'application doit tourner au premier plan (ex: `node server.js` ou `nginx -g "daemon off;"`). *Si vous avez besoin de plusieurs services, utilisez **Docker Compose** avec un conteneur par service.*

3. Comment fonctionne un Service ?

Les services sont définis par des fichiers "Unit" (ex: `mon-app.service`) situés dans `/etc/systemd/system/`.

Exemple de script (Unit File) :

```
[Unit]
Description=Mon Application Node.js
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/var/www/mon-app
ExecStart=/usr/bin/node server.js
Restart=on-failure
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

3. Les champs d'un Service (1/2)

La section `[Unit]` : Métadonnées et Dépendances

- `Description` : Le nom affiché dans `systemctl status`.
- `After` / `Before` : Gère l'ordre (ex: `After=network.target` attend le réseau).
- `Requires` : Dépendance forte. Si le service ciblé plante, notre service s'arrête.
- `Wants` : Dépendance faible. Tente de démarrer le service ciblé, mais continue s'il échoue.

La section `[Install]` : Activation au Démarrage

- `WantedBy` : Définit dans quel contexte (runlevel) l'activer.
  - *`multi-user.target`* : Standard pour activer un service au démarrage du système.

3. Les champs d'un Service (2/2)

La section `[Service]` : Exécution et Comportement

- `Type` : `simple` (par défaut), `forking` (démons traditionnels), `notify`.
- `User` / `Group` : Le propriétaire du processus (règle d'or : éviter de tourner en `root`).
- `WorkingDirectory` : Le dossier depuis lequel la commande sera lancée (comme un `cd`).
- `ExecStart` / `ExecStop` : Les commandes de lancement/arrêt (utiliser les **chemins absolus**).
- `EnvironmentFile` : Permet de charger un fichier `.env` contenant vos secrets.
- `Restart` : Conditions de redémarrage (`always`, `on-failure`, `on-abnormal`...).
- `RestartSec` : Temps d'attente avant de redémarrer le service après un crash (ex: `5s`).

4. Sécurité Systemd (1/2)

En tant qu'Admin Sys, vous ne devez **jamais** faire confiance à une application.

1. Le Moindre Privilège

- **Le Risque :** Si votre application NodeJS tourne en `root` et se fait pirater, le pirate a le contrôle total de votre serveur.
- **La Solution :** Toujours créer un utilisateur dédié sans mot de passe et l'assigner au service.

```
[Service]
User=nodeapp
Group=nodeapp
```

2. Isolation du Système

- **Le Risque :** Une application compromise pourrait tenter de modifier les fichiers vitaux de Linux (`/boot` , `/usr`).
- **La Solution :** Rendre l'OS en lecture seule pour le service.

```
[Service]
ProtectSystem=strict
```

4. Sécurité Systemd (2/2)

Il faut également protéger les autres utilisateurs et empêcher l'escalade de privilèges.

3. Protection des Utilisateurs

- **Le Risque :** L'application lit les dossiers personnels (`/home/karl`) pour y voler des clés SSH.
- **La Solution :** Interdire l'accès aux dossiers `/home` et `/root` .

```
[Service]
ProtectHome=yes
```

4. Pas de Nouveaux Privilèges

- **Le Risque :** L'application utilise la commande `sudo` ou un binaire `setuid` pour s'élever.
- **La Solution :** Bloquer toute escalade de privilèges possible.

```
[Service]
NoNewPrivileges=yes
```

5. TP Basique : Créer un service robuste (1/2)

Objectif : Créer un service capable de gérer une panne et de redémarrer automatiquement.

1. **Créer le script bash (`/opt/hello.sh`)**
   - Écrivez une boucle infinie (`while true`) qui affiche un message et la date, puis attend 5 secondes.
   - À l'intérieur de la boucle, si le fichier `/opt/viral-failure` existe, le script doit s'arrêter en erreur (`exit 1`).
   - N'oubliez pas de rendre le fichier exécutable.

2. **Créer le fichier de service (`hello.service`)**
   - Créez le fichier `/etc/systemd/system/hello.service`.
   - Configurez l'exécution de votre script.
   - Configurez le redémarrage automatique en cas d'échec.
   - **Important :** Ajoutez un délai d'attente de 15 secondes avant le redémarrage.

5. TP Basique : Tester la panne (2/2)

3. **Activer et démarrer le service**
   - Rechargez systemd, démarrez le service et observez les logs en temps réel.

4. **Simulation de panne (L'épreuve du feu)**
   - Pendant que les logs défilent, créez le fichier `/opt/viral-failure`.
   - Observez le crash du service.
   - Pendant le délai de 15 secondes, supprimez vite le fichier.
   - Observez le service ressusciter et reprendre son fonctionnement normal !

6. Cheat Sheet : Systemd

📁 Fichiers & Logs

DOSSIER DES SERVICES

```
/etc/systemd/system/
```

VOIR TOUS LES LOGS

```
journalctl -u mon-app
```

LOGS EN TEMPS RÉEL (FOLLOW)

```
journalctl -u mon-app -f
```

💻 Commandes CLI

RECHARGER LES FICHIERS (.SERVICE)

```
systemctl daemon-reload
```

ACTIVER AU DÉMARRAGE

```
systemctl enable mon-app
```

GESTION COURANTE

```
systemctl start/stop/status
```

⚙️ Unit File (.service)

`ExecStart=`

Commande à exécuter (chemin absolu !).

`Restart=on-failure`

Redémarre si le code de sortie n'est pas 0.

`User= / Group=`

L'utilisateur qui lance le processus.

Module Crontab

Planifier des tâches comme un pro

AU PROGRAMME

- Comprendre l'utilité du démon Cron
- Maîtriser la syntaxe de planification temporelle
- Connaître les pièges classiques et savoir les éviter
- TP : Multi-Tâches

1. Introduction : Le démon Crontab : Automatiser l'Administration

Qu'est-ce que c'est ? Le démon `cron` est un planificateur de tâches historique (et incontournable) sous Linux. Il permet d'exécuter des scripts, des commandes ou des programmes en arrière-plan à des moments précis.

À quoi ça sert ?

- **Sauvegardes** : Lancer un dump de base de données toutes les nuits à 3h du matin.
- **Maintenance** : Vider les fichiers temporaires ou archiver des logs chaque semaine.
- **Monitoring** : Exécuter un script qui vérifie la santé d'un service toutes les 5 minutes.
- **Synchronisation** : Récupérer des données externes (API, FTP) à intervalle régulier.

2. Installation de Cron

Bien que très courant, `cron` n'est pas toujours installé par défaut sur des images minimalistes (comme les conteneurs Docker légers).

Comment l'installer ? (Debian/Ubuntu)

```
sudo apt update
sudo apt install cron
```

Démarrer le démon : Une fois installé, il faut s'assurer que le service qui exécute les tâches tourne bien en arrière-plan.

```
# Vérifier le statut
systemctl status cron

# Démarrer le service s'il est inactif
systemctl start cron

# L'activer au démarrage du serveur
systemctl enable cron
```

2. Comment ça fonctionne ?

On édite les tâches d'un utilisateur via la commande : `crontab -e` (ou `sudo crontab -e` pour le root). Pour lister ses tâches actives : `crontab -l` .

La Syntaxe Magique (5 astérisques)

```
* * * * * [user] commande à exécuter
| | | | |
| | | | +——— Jour de la semaine (0 - 7) (Dimanche = 0 ou 7)
| | | +——— Mois (1 - 12)
| | +——— Jour du mois (1 - 31)
| +——— Heure (0 - 23)
+——— Minute (0 - 59)
```

Exemples de lecture :

- `30 2 * * * /opt/backup.sh` : Tous les jours à 02h30. Exécuté par l'utilisateur 'root'.
- `*/5 * * * * karl /opt/check.sh` : Toutes les 5 minutes. Exécuté par l'utilisateur 'karl'.
- `0 0 * * 1 /opt/clean.sh` : Tous les lundis à minuit exactement. Exécuté par l'utilisateur 'root'.

3. Erreurs fréquentes & Pièges (1/2)

1. Le piège de l'environnement (PATH) Cron s'exécute dans un environnement très limité (souvent `PATH=/usr/bin:/bin`). Il ne connait pas vos alias ni certains de vos logiciels. *Solution :* Toujours utiliser les **chemins absolus** pour vos commandes et vos scripts (`/usr/local/bin/node` plutôt que `node`).

2. Les sorties (STDOUT / STDERR) silencieuses Si votre script affiche du texte ou des erreurs, Cron essaiera d'envoyer un email à l'utilisateur (ce qui échoue souvent et spamme le système). *Solution :* Rediriger les sorties vers un fichier log. `* * * * * /opt/script.sh >> /var/log/mon-script.log 2>&1`

3. Oublier de rendre le script exécutable Un classique ! N'oubliez pas le `chmod +x /opt/script.sh` .

4. La ligne vide à la fin du fichier Certaines versions de `cron` exigent qu'il y ait un retour à la ligne (Enter) à la toute fin du fichier de configuration, sinon la dernière règle est simplement ignorée.

4. Sécurité Crontab

L'automatisation est un vecteur d'attaque majeur si elle est mal configurée.

🛡️ 1. Contrôle d'Accès (Allow/Deny)

- **Le Risque :** N'importe quel utilisateur sur le serveur (ex: `www-data` piraté) peut créer des tâches récurrentes pour persister son piratage.
- **La Solution :** Restreindre l'utilisation de la commande `crontab`.
- **Fichiers clés :**
  - `/etc/cron.deny` : Liste noire (les utilisateurs dedans ne peuvent pas utiliser cron).
  - `/etc/cron.allow` : Liste blanche (si ce fichier existe, **seuls** les utilisateurs dedans peuvent utiliser cron, c'est l'approche la plus sécurisée !).

🛡️ 2. Le Piège de l'Escalade de Privilèges

- **Le Risque :** Vous planifiez `/opt/backup.sh` en tant que `root`. Mais ce fichier appartient à l'utilisateur `karl` ou a des droits d'écriture permissifs (`chmod 777`).
- **L'Exploitation :** L'utilisateur normal `karl` modifie le contenu de `backup.sh` pour y insérer une commande malveillante. À minuit, `root` exécutera cette commande à l'aveugle !
- **La Solution :** Tout script exécuté par le cron de `root` **doit appartenir à root** et ne **jamais** être modifiable par un tiers (`chmod 755` ou `700`).

5. TP Basique : Multi-Tâches

Objectif : Créer un monitoring simple et archiver ses logs régulièrement.

1. **Tâche 1 : Monitoring (`/opt/monitor.sh`)**
   - Script qui affiche la date et la commande `uptime`.
   - Planifiez son exécution **toutes les minutes** avec redirection vers `/var/log/monitor.log`.
2. **Tâche 2 : Rotation des logs (`/opt/rotate.sh`)**
   - Script qui copie `monitor.log` vers une archive horodatée (ex: `monitor-YYYYMMDD_HHMM.log`) puis vide le log principal.
   - Le script doit prendre en **argument** le nombre maximum d'archives à conserver et supprimer les plus anciennes.
   - Planifiez-le **tous les jours à minuit** (toutes les 5 minutes pour tester).

6. Cheat Sheet : Crontab

📁 Fichiers Clés

FICHIERS DES UTILISATEURS

```
/var/spool/cron/crontabs/
```

CRON SYSTÈME GLOBAL

```
/etc/crontab
```

LOGS D'EXÉCUTION

```
/var/log/syslog
```

💻 Commandes CLI

ÉDITER SA CRONTAB

```
crontab -e
```

VOIR SA CRONTAB (LISTE)

```
crontab -l
```

ÉDITER POUR QQN D'AUTRE (ROOT)

```
crontab -u karl -e
```

⚙️ Syntaxe du Temps

```
* * * * *
```

MIN HEURE JOUR MOIS JOURSEM

```
>> /var/log/app.log 2>&1
```

À TOUJOURS AJOUTER À LA FIN (Logs et Erreurs).

Module NGINX

De la Requête à la Configuration

AU PROGRAMME

- Maîtriser l'installation et la gestion
- Comprendre le cycle d'une requête HTTP
- TP : Serveur Statique
- TP : Reverse Proxy

1. NGINX : Le Couteau Suisse du Web

Qu'est-ce que NGINX ? NGINX (prononcé *Engine-X*) est un serveur web open-source extrêmement performant, créé pour résoudre le problème de la gestion de milliers de connexions simultanées (le problème C10k).

Ses rôles principaux :

- **Serveur HTTP statique** : Distribuer des fichiers (HTML, CSS, images) à très haute vitesse.
- **Reverse Proxy** : Protéger et répartir la charge devant des serveurs d'application (Node.js, Python, PHP).
- **Load Balancer** : Répartir le trafic entre plusieurs serveurs pour assurer la haute disponibilité.

Il est aujourd'hui le serveur web le plus utilisé au monde devant Apache.

2. Installation & Premiers Pas

Déploiement sur distribution **Debian/Ubuntu**.

- **Installation rapide :**

```
sudo apt update && sudo apt install nginx
```

- **Vérification du service :**

```
systemctl status nginx
```

- **Commandes de gestion :**
  - `systemctl start nginx` : Lancer le service
  - `systemctl stop nginx` : Arrêter le service
  - `systemctl reload nginx` : Appliquer la configuration sans coupure
  - `systemctl restart nginx` : Redémarrer complètement le service

2. Architecture NGINX

Arborescence dans `/etc/nginx/`

Le Cœur & Les Brouillons

- **nginx.conf** : Fichier de configuration global (processus, logs).
- **sites-available/** :
  - Dossier des "brouillons".
  - Contient tous vos fichiers de configuration.
  - Les sites ici ne sont **pas encore actifs**.

La Production Active

- **sites-enabled/** :
  - Dossier des sites en ligne.
  - Ne contient que des **liens symboliques**.
- **Lien symbolique (** `ln -s` **)** : Permet d'activer/désactiver un site sans supprimer le fichier original.

2. Architecture (Sécurité)

Précautions indispensables avant toute mise en production.

- **Test de syntaxe :** Interdiction de redémarrer sans tester !

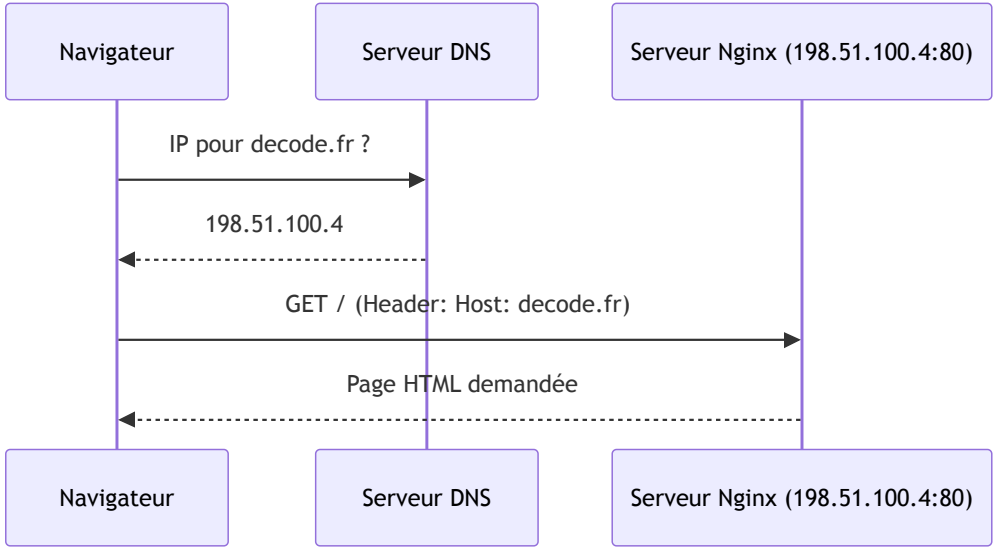
```
nginx -t
```

- **Rechargement à chaud :** Applique les changements sans couper les connexions en cours.

```
systemctl reload nginx
```

3. La Négociation HTTP (1/2)

Que se passe-t-il **AVANT** qu'Nginx ne traite la demande ?



Le secret du Host :

La requête contient obligatoirement le Header HTTP `Host` (ex: `Host: decode.fr`).

C'est cet en-tête précis qui permet au serveur NGINX d'identifier quel site web afficher parmi tous ceux qui sont hébergés sur la même machine.

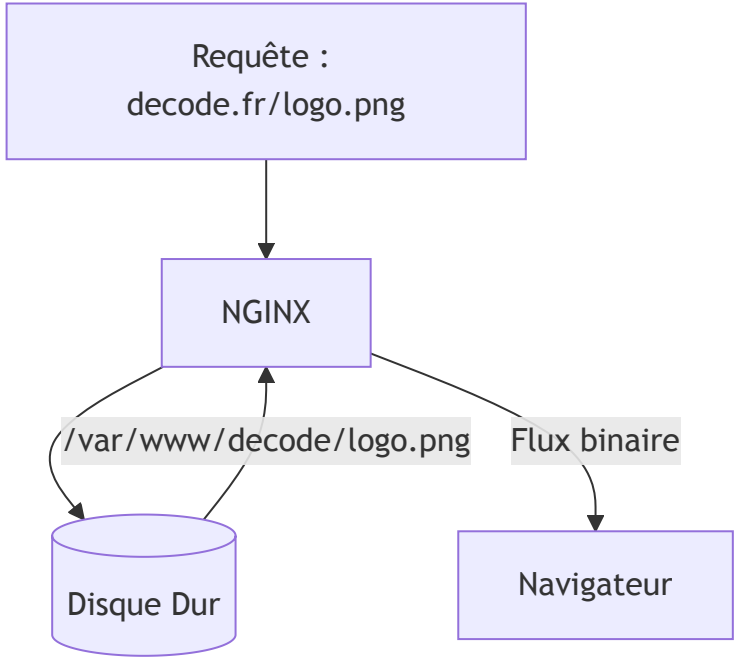
3. La Négociation HTTP (2/2)

Le fonctionnement des "Virtual Hosts" :

- **Port unique (80/443) :** Toutes les requêtes HTTP/HTTPS entrantes frappent la même adresse IP et le même port serveur.
- **Multiplexage :** NGINX est conçu pour héberger et servir des dizaines de sites web différents sur cette unique adresse IP.
- **Lecture de l'en-tête :** À la réception de la requête, NGINX extrait immédiatement la valeur du Header `Host`.
- **Routage interne :** Il parcourt l'ensemble de ses fichiers de configuration à la recherche d'une directive `server_name` correspondante et exécute le code du bloc trouvé.

4. Concept : Serveur Statique

Comment NGINX délivre-t-il un fichier web ?



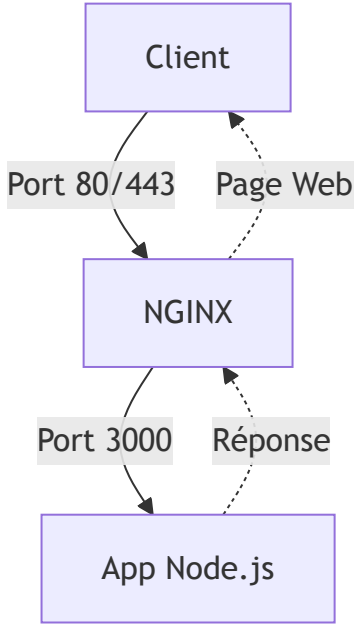
Le Flux Statique

- NGINX agit comme un passe-plat ultra-rapide.
- Il convertit directement l'URL demandée en un chemin de fichier sur le disque dur.
- Il n'y a **aucun calcul**. La performance dépend uniquement de la vitesse du réseau et du disque.

4. Concept : Le Reverse Proxy

Pourquoi utiliser un Reverse Proxy ?

- **Le problème :** Nginx ne sait pas lire du code (Node.js, PHP).
- **La solution :** Nginx agit comme un **bouclier frontal** qui passe la main à un serveur d'application local via le réseau interne.
- **Avantage :** Nginx gère le SSL, la sécurité et la compression.



5. Configuration NGINX (Bases)

Comment dire à NGINX quoi faire avec une requête ?

Le bloc `server {}`

Définit un "hôte virtuel". Il indique pour **quel domaine** et **quel port** cette configuration s'applique.

```
server {
    listen 80;
    server_name decode.fr;

    # ... règles spécifiques
}
```

Le bloc `location {}`

Définit comment traiter les **chemins spécifiques** de l'URL à l'intérieur de ce serveur.

```
location / {
    # Si on demande la racine
    root /var/www/html;
}

location /assets/ {
    # Un dossier spécifique
    root /var/www/static;
}
```

5. Configuration NGINX (Proxy)

Le Reverse Proxy nécessite de transmettre l'identité de l'utilisateur au serveur applicatif, sinon ce dernier croira que toutes les requêtes proviennent de `127.0.0.1`.

```
location /api/ {
    # 1. Transfert de la requête vers l'application locale
    proxy_pass http://127.0.0.1:3000;

    # 2. En-têtes classiques (Toujours massivement utilisés)
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    # 3. Nouveau standard officiel (RFC 7239)
    proxy_set_header Forwarded "for=$remote_addr;proto=$scheme;host=$host";
}
```

5. HTTP Proxy vs FastCGI

NGINX peut dialoguer avec l'application backend de deux manières différentes.

🌐 `proxy_pass` (Standard HTTP)

- **Usage :** Node.js, Python, Ruby, Go.
- **Fonctionnement :** NGINX agit comme un client web. Il refait une requête **HTTP complète** vers le port du backend.
- **Le Backend :** Doit être un véritable serveur web autonome capable de comprendre le protocole HTTP.

```
location / {
    proxy_pass http://127.0.0.1:3000;
}
```

⚡ `fastcgi_pass` (Binaire)

- **Usage :** Principalement **PHP** (via `php-fpm`).
- **Fonctionnement :** NGINX utilise le protocole FastCGI (plus rapide et léger). Il n'envoie pas de HTTP mais des variables brutes.
- **Le Backend :** Ce n'est pas un serveur web, c'est juste un moteur d'exécution (FPM) sur un Socket Unix.

```
location ~ \.php$ {
    include fastcgi.conf;
    fastcgi_pass unix:/run/php/php8.2-fpm.sock;
}
```

5. Configuration (Cas Avancés)

Deux situations incontournables pour héberger des applications modernes.

🌐 Le Fallback SPA (React, Vue)

- **Le Problème :** Sur une Single Page App, NGINX renvoie une erreur `404` si on actualise une page spécifique (ex: `/dashboard`), car ce fichier n'existe pas physiquement sur le disque.
- **La Solution :** Forcer NGINX à rediriger toutes les requêtes introuvables vers le routeur frontend (`index.html`).

```
location / {
    root /var/www/mon-app;
    # Tente le fichier, sinon le dossier, sinon inde
    try_files $uri $uri/ /index.html;
}
```

🛡️ L'Isolation (Symfony, Laravel)

- **Le Problème :** Un framework possède souvent un fichier `.env` sensible à sa racine. NGINX ne doit **jamais** exposer la racine complète du projet !
- **La Solution :** Restreindre la directive `root` au sous-dossier `public/`. Cela crée une sorte de prison interdisant au navigateur de lire le code source.

```
server {
    listen 80;
    server_name api.decode.fr;

    # On isole la racine du web au dossier public !
    root /var/www/mon-api/public;
}
```

6. Erreurs & Pièges (1/3)

Les erreurs classiques de l'administrateur système débutant.

❌ 403 Forbidden (Statique)

- **Le Symptôme :** NGINX affiche "403 Forbidden" au lieu du site web.
- **La Réalité :** C'est un **problème de permissions Linux**. L'utilisateur interne d'NGINX (`www-data`) n'a pas les droits de lecture sur le dossier `/var/www/` .

❌ 502 Bad Gateway (Proxy)

- **Le Symptôme :** Une page d'erreur blanche affichant "502".
- **La Réalité :** NGINX va très bien ! C'est l'application cible (ex: sur le port 3000) qui est **éteinte ou plantée**. NGINX frappe à une porte fermée.

6. Erreurs & Pièges (2/3)

Pièges de configuration et bonnes pratiques.

❌ L'Oubli des Headers

- **Le Symptôme :** L'application backend voit toutes les requêtes provenir de l'IP `127.0.0.1`.
- **Le Piège :** Utiliser `proxy_pass` seul. Le proxy agit en son nom et masque le véritable utilisateur.
- **La Solution :** Toujours faire suivre l'identité du client avec des directives comme `proxy_set_header X-Real-IP $remote_addr;`.

❌ Le Crash en Production

- **Le Piège :** Modifier un fichier de config et relancer brutalement avec `systemctl restart nginx`. S'il manque un point-virgule, le service refuse de démarrer et **le site coupe**.
- **La Règle d'Or :** Toujours faire un `nginx -t` avant de relancer ou de recharger (`reload`).

6. Erreurs & Pièges (3/3)

Attention à la logique de routage d'NGINX !

❌ Le Fallback Alphabétique

- **Le Piège :** NGINX reçoit une requête mais ne trouve **aucun** bloc `server_name` qui correspond.
- **La Réalité :** Il va renvoyer la requête sur **le premier fichier de configuration chargé, par ordre alphabétique** !
- **La Solution :** Supprimer le fichier `default` préinstallé qui intercepte souvent ce trafic : `sudo rm /etc/nginx/sites-enabled/default`.

❌ Le Port 80 Bloqué

- **Le Piège :** Connexion refusée depuis l'extérieur (le navigateur mouline puis affiche une erreur).
- **Causes fréquentes :**
  - Le pare-feu bloque l'accès (`ufw allow 'Nginx Full'`).
  - Un autre logiciel (comme Apache2) tourne déjà et monopolise le port 80.

7. Sécurité NGINX (1/2)

La sécurité n'est pas une option. Voici les standards minimaux à appliquer :

1. Masquer la Version

- **Le Risque :** NGINX affiche sa version exacte (ex: `1.24.0`) sur les pages d'erreur. Les attaquants s'en servent pour exploiter des failles connues.
- **La Solution :** Dans `/etc/nginx/nginx.conf` (contexte `http`) :

```
server_tokens off;
```

2. Bloquer les Fichiers Sensibles

- **Le Risque :** Le téléchargement accidentel de vos fichiers `.env` ou du dossier `.git`.
- **La Solution :** Bloquer tout ce qui commence par un point.

```
location ~ /\. {
    deny all;
}
```

7. Sécurité NGINX (2/2)

Il faut également sécuriser les navigateurs de vos visiteurs.

🛡️ 3. Les En-têtes de Sécurité

- **Le Risque :** Le Clickjacking (votre site dans une iframe piégée) ou les failles MIME.
- **La Solution :** Ajouter ces headers globaux :

```
# Empêcher l'intégration dans une iframe externe
add_header X-Frame-Options "SAMEORIGIN";

# Bloquer le reniflage de type MIME
add_header X-Content-Type-Options "nosniff";
```

8. TP : Serveur Statique

Objectif : Héberger un mini-site web (HTML) accessible depuis le navigateur.

1. **Préparation des fichiers**
   - Créez un dossier `/var/www/decode`.
   - Créez un fichier `index.html` à l'intérieur avec un mot de bienvenue.
2. **Configuration NGINX**
   - Créez un nouveau bloc de configuration (`server`) dans NGINX.
   - Il doit écouter sur le port `80`.
   - Il doit utiliser le nom de domaine `ecole-decode.local` (`server_name`).
   - La racine (`root`) doit pointer vers votre nouveau dossier.

8. TP : Reverse Proxy

Objectif : Rediriger le trafic NGINX vers une application locale tournant sur un autre port.

1. **Lancement de l'application locale**
   - Imaginez qu'une application (ex: Node.js, PHP) tourne sur le port `3000` de votre serveur.
2. **Configuration NGINX**
   - Dans votre bloc de configuration, utilisez `proxy_pass` pour rediriger le trafic racine `/` vers `http://127.0.0.1:3000`.
   - Transmettez les informations d'origine au backend en configurant les en-têtes `Host` et `X-Real-IP`.
3. **Bonus : Variables d'environnement**
   - Par défaut, NGINX ne comprend pas les variables d'environnement dans sa configuration.
   - Il vous faudra installer l'utilitaire `envsubst` via le paquet `gettext-base`.
   - Créez un fichier template contenant des variables (ex: `${PORT}`, `${HOSTNAME}` et `${SERVER_URL}`).
   - Utilisez la commande Linux `envsubst` pour générer le fichier de configuration final.

9. Cheat Sheet : NGINX

📁 Fichiers Clés

CONFIGURATION GLOBALE

```
/etc/nginx/nginx.conf
```

CRÉER UN SITE

```
/etc/nginx/sites-available/
```

ACTIVER UN SITE (SYMLINK)

```
/etc/nginx/sites-enabled/
```

💻 CLI NGINX

VÉRIFIER LA SYNTAXE (VITAL)

```
nginx -t
```

RECHARGER (SANS COUPURE)

```
systemctl reload nginx
```

ÉTAT DU SERVICE

```
systemctl status nginx
```

⚙️ Directives

```
server { }
```

Définit un hôte virtuel (domaine, port).

```
location / { }
```

Intercepte une route URL spécifique.

```
proxy_pass http:// ...
```

Transfère le trafic (Reverse Proxy).

Module PHP & PHP-FPM

Architecture, Installation et Configuration

AU PROGRAMME

- PHP Classique vs PHP-FPM
- Installation (FPM, Modules, PDO)
- Architecture : NGINX & FastCGI
- Configuration : php.ini & pools
- Erreurs courantes (502, sockets)
- Sécurité : Bonnes pratiques
- TP : Connexion NGINX & PHP-FPM

1. PHP Classique vs PHP-FPM

Historiquement, PHP était souvent exécuté via un module Apache (`mod_php`) ou en tant que script CGI. Ces méthodes, bien que simples, présentaient des limites en termes de scalabilité et de performances.

La Révolution PHP-FPM (FastCGI Process Manager) PHP-FPM est une implémentation alternative de FastCGI avec des fonctionnalités supplémentaires hautement optimisées pour les sites très fréquentés.

PHP Classique (mod_php)

- Intégré directement dans le serveur web (Apache).
- Exécute PHP sous le même utilisateur que le serveur web.
- Lourd : Le serveur web charge le moteur PHP pour **chaque** requête (même pour une image).
- Non compatible avec NGINX de manière native.

PHP-FPM (FastCGI)

- Service **indépendant** (démon) tournant en arrière-plan.
- Gère son propre pool de processus (workers).
- Le serveur web (NGINX/Apache) "délègue" uniquement les requêtes `.php` via un socket.
- Ultra performant, consommation mémoire optimisée.

2. Installation de PHP-FPM & Modules

1. Installation Core

L'installation de PHP-FPM est standardisée via `apt` sur Debian/Ubuntu.

```
# Met à jour les paquets
apt update

# Installe PHP-FPM
apt install php-fpm
```

PHP-FPM devient un **service systemd** :

```
systemctl status php8.2-fpm
```

2. Le Système de Modules

PHP a besoin de modules (bibliothèques dynamiques) pour interagir avec des bases de données ou des APIs.

```
# PDO PostgreSQL
apt install php-pgsql

# Requêtes HTTP (APIs)
apt install php-curl

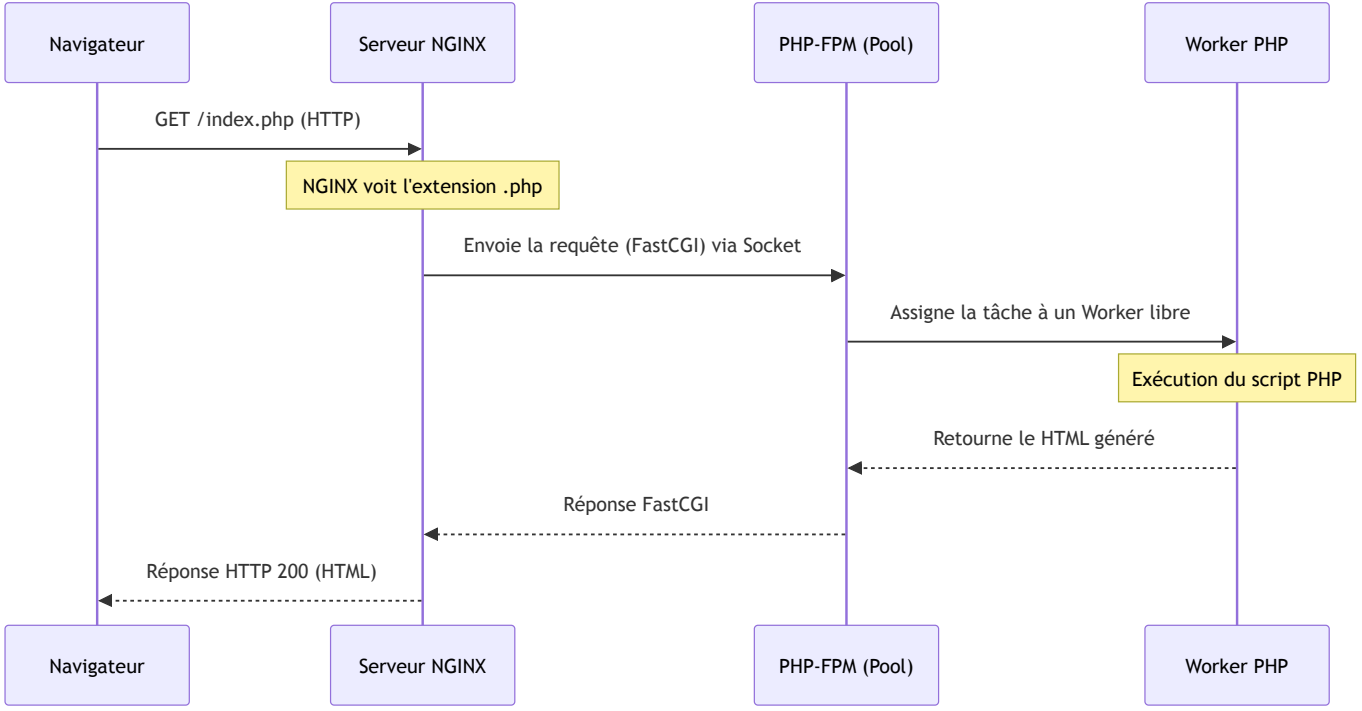
# Compression
apt install php-zip php-mbstring
```

Après l'installation d'un module, il faut **toujours** redémarrer FPM :

`systemctl restart php8.2-fpm`

3. Architecture NGINX & FastCGI

Pour que NGINX (qui ne comprend que du HTTP statique) puisse exécuter du code PHP, il doit se comporter comme un **Reverse Proxy** vers PHP-FPM, en utilisant le protocole **FastCGI**.



4. Configuration : Le fichier php.ini

Le fichier `php.ini` configure le comportement global du langage PHP.

Attention, Debian/Ubuntu gère **deux configurations séparées** !

- `/etc/php/8.x/fpm/php.ini` : Appliquée aux requêtes web (via NGINX).
- `/etc/php/8.x/cli/php.ini` : Appliquée quand vous utilisez `php` dans le terminal.

Modifier l'un n'affecte pas l'autre !

Paramètres communs cruciaux (`fpm/php.ini`) :

```
; Limite mémoire par script (Attention aux fuites de mémoire)
memory_limit = 256M

; Fuseau horaire par défaut
date.timezone = Europe/Paris

; Tailles maximales des uploads
upload_max_filesize = 10M
post_max_size = 50M
max_file_uploads = 5
```

4. Configuration : Le Pool FPM

Le comportement du démon PHP-FPM en tant que tel est configuré séparément via des "pools". Le pool par défaut s'appelle `www`.

Fichier de configuration du Pool : `/etc/php/8.x/fpm/pool.d/www.conf`

```
; Utilisateur sous lequel tournent les scripts PHP (Doit correspondre aux droits NGINX)
user = www-data
group = www-data

; Adresse d'écoute (ici un socket UNIX)
listen = /run/php/php8.2-fpm.sock
; Alternative : écoute sur un port TCP/IP (si NGINX est sur un autre serveur/conteneur)
; listen = 127.0.0.1:9000

; --- Gestion des processus (workers) ---
pm = dynamic              ; Le nombre de processus s'adapte à la charge
pm.max_children = 50      ; Nombre maximum absolu de workers simultanés
pm.start_servers = 5      ; Nombre de workers démarrés au lancement du service
pm.min_spare_servers = 5  ; Garde toujours au moins 5 workers inactifs sous la main
pm.max_spare_servers = 35 ; Tue les workers inactifs s'il y en a plus de 35
```

5. Erreurs Courantes & Pièges

❌ "502 Bad Gateway"

L'erreur NGINX `502` avec PHP-FPM signifie que NGINX n'arrive pas à communiquer avec le Socket FPM.

- **Cause 1** : Le service `php-fpm` est arrêté/planté.
- **Cause 2** : Mauvais chemin dans la conf NGINX (`fastcgi_pass`).
- **Cause 3** : Problème de **droits** (`listen.owner` / `listen.group`).

❌ Oubli du redémarrage

Modifier `php.ini` n'a **aucun effet immédiat**. Le démon tourne en permanence en arrière-plan.

Il faut toujours redémarrer les workers :

```
systemctl reload php8.2-fpm
```

L'upload dépend de **deux configurations distinctes** :

- **1. Côté NGINX :** Par défaut, bloque tout envoi > 1Mo (Erreur `413 Payload Too Large`). Il **faut** ajouter `client_max_body_size 50M;` dans NGINX.
- **2. Côté PHP (`php.ini`) :** La taille globale POST DOIT être supérieure au poids max d'un fichier × le nombre de fichiers autorisés. *Règle d'or :* `post_max_size` ≥ (`upload_max_filesize` × `max_file_uploads`).

*Ex: Si `upload_max_filesize = 10M` et `max_file_uploads = 5` , alors NGINX **et** `post_max_size` doivent être à 50M !*

6. Sécurité et Bonnes Pratiques

PHP est puissant, mais cette puissance le rend vulnérable si le serveur n'est pas "durci".

🛡️ 1. Masquer l'identité (`php.ini`)

Par défaut, PHP envoie un en-tête HTTP `X-Powered-By: PHP/8.x.x` dans chaque réponse, ce qui révèle votre version exacte aux attaquants.

```
1   expose_php = Off
```

🛡️ 2. Bloquer les commandes système (`php.ini`)

Certaines fonctions PHP natives permettent d'interagir directement avec le terminal du serveur. Si votre code n'en a pas besoin, bloquez-les.

```
1   disable_functions = exec, shell_exec, system ...
```

🛡️ 3. Isolation des droits (`www.conf`)

- **Droit d'exécution :** Ne lancez **JAMAIS** le pool FPM en `root`. Assurez-vous que les workers s'exécutent avec des droits limités (`user = www-data`).
- **Cloisonnement (Hébergement) :** Si vous hébergez plusieurs sites (ex: Client A et Client B), créez **plusieurs pools FPM** (`clientA.conf` , `clientB.conf`) avec des utilisateurs Linux différents pour chaque pool.
- **Résultat :** Si le site du Client A est piraté, l'attaquant ne pourra pas lire ou modifier les fichiers du Client B !

7. TP : Servir une page PHP

Objectif : Configurer NGINX pour relayer le traitement PHP à FPM.

1. **Étape 1 : Préparation du fichier PHP**
   - Créez un dossier `/var/www/php`.
   - Créez un fichier `info.php`.
   - À l'intérieur, écrivez le code minimal permettant d'appeler la fonction interne `phpinfo();` pour afficher la configuration de PHP.

2. **Étape 2 : Configuration du Server Block NGINX**
   - Créez un fichier de configuration NGINX compatible PHP-FPM (ex: `/etc/nginx/sites-available/php.conf`).
   - Configurez les paramètres suivants :
     - Timezone en Europe/Paris.
     - La possibilité d'upload 10 fichiers de 3 Mo maximum chacun.
     - Le site doit être accessible à l'adresse `http://monsitephp.local`

8. Cheat Sheet PHP-FPM

💻 Commandes FPM

INSTALLATION GLOBALE

```
apt install php-fpm php-mysql
```

LISTER LES MODULES

```
php -m
```

RECHARGER (SANS COUPURE)

```
systemctl reload php8.2-fpm
```

LIRE LES LOGS EN DIRECT

```
tail -f /var/log/php8.2-fpm.log
```

📁 Fichiers Clés

CONFIG GLOBALE FPM

```
/etc/php/8.x/fpm/php.ini
```

CONFIG TERMINAL (CLI)

```
/etc/php/8.x/cli/php.ini
```

CONFIG POOL WORKERS

```
/etc/php/8.x/fpm/pool.d/www.conf
```

SOCKET UNIX

```
/run/php/php8.x-fpm.sock
```

⚙️ Snippet NGINX

CAPTURE LES REQUÊTES PHP VIA REGEX.

```
location ~ \.php$ { }
```

CHARGE LES PARAMÈTRES FASTCGI STANDARDS.

```
include snippets/fastcgi-php.conf;
```

TRANSFÈRE LA REQUÊTE AU SOCKET PHP-FPM.

```
fastcgi_pass unix:/ … /php8.x-fpm.sock;
```